

DOCUMENTACIÓN TÉCNICA DEL PROYECTO

Entorno de Formación en Centros de Trabajo (FCT)

Este documento contiene la especificación arquitectónica, el diseño de la base de datos relacional, la definición de endpoints de la API RESTful y las directrices técnicas del ecosistema de software desarrollado para la gestión integral de citas automatizadas.

Autores / Alumnos:	Rafael Jesús Martínez Moreno Marcos Pozo Amador
Ciclo Formativo:	Técnico Superior en Desarrollo de Aplicaciones Web (DAW)
Centro Educativo:	IES Pedro Jiménez Montoya
Empresa Colaboradora:	Impulsa Telecom
Tutora de Empresa:	Cristina Latorre Aparicio
Profesora de Seguimiento:	Ángeles Román Rodríguez
Fecha de Cierre:	Mayo de 2026

1. Descripción General del Sistema

AppCitas es una plataforma web integral diseñada bajo un modelo multitenant (multinegocio) que permite la coexistencia y administración independiente de múltiples comercios de servicios dentro de un único ecosistema de software. El sistema provee flujos diferenciados para tres actores principales: el cliente final (quien agenda, confirma o cancela citas de manera autónoma), el administrador del negocio (responsable del catálogo de servicios, horarios de empleados, bloqueos de agenda y personalización visual) y el superadministrador (con capacidades globales para dar de alta comercios y asignar gestores).

1.1 Arquitectura del Software

La aplicación implementa un desacoplamiento absoluto mediante una arquitectura cliente-servidor nativa, estructurada en dos repositorios core:

- **Backend (NestJS):** Se basa en una arquitectura modular orientada a dominios que encapsula las responsabilidades de disponibilidad, lógica de reservas, subida de ficheros y auditoría. Sigue principios de inyección de dependencias para asegurar la mantenibilidad y un acoplamiento laxo.
- **Frontend (React):** Diseñado como una Single Page Application (SPA), gestiona el estado dinámico mediante contextos globales y flujos declarativos de renderizado sobre componentes atómicos reutilizables y un enrutamiento seguro del lado del cliente.

2. Stack Tecnológico

El núcleo tecnológico ha sido seleccionado bajo criterios de robustez, tipado seguro y alto rendimiento para la concurrencia de transacciones:

Tecnologías del Backend

- **NestJS (Node.js):** Framework empresarial configurado bajo formato modular para proporcionar robustez, interceptores globales y un sistema ágil de Guards de seguridad.
- **PostgreSQL:** Motor de base de datos relacional elegido para garantizar la integridad referencial y soportar consultas complejas de concurrencia temporal.
- **JWT (JSON Web Tokens):** Mecanismo de autenticación sin estado para proteger las rutas privadas de administración.
- **Nodemailer & Ethereal:** Suite de transporte SMTP para la renderización y entrega distribuida de notificaciones multiparte (HTML estricto y texto plano).
- **Multer:** Middleware especializado en el procesamiento de flujos de archivos binarios para la subida controlada de logos.

- **Joi:** Motor de validación en tiempo de ejecución para asegurar que los payloads de entrada cumplen esquemas restrictivos antes de impactar el controlador.

Tecnologías del Frontend

- **React (v18+):** Biblioteca basada en componentes para la interfaz de usuario con hooks personalizados para el aislamiento del estado de los formularios.
- **React Router:** Gestor de navegación declarativa con soporte para la protección y redirección automática de rutas no autenticadas.
- **Bootstrap & React Bootstrap:** Framework de diseño adaptativo para garantizar la correcta visualización responsive del panel en smartphones y ordenadores sin sobrecarga de CSS.

3. Diseño de la Base de Datos Relacional

La base de datos se aloja en PostgreSQL, utilizando identificadores únicos universales (**UUID**) en lugar de claves secuenciales enteras para prevenir ataques de enumeración y asegurar la escalabilidad del almacenamiento multinegocio.

Todas las tablas clave implementan un mecanismo de persistencia histórica mediante **Soft Delete** (controlado por la columna indexada **deleted_at**), permitiendo la recuperación de registros y manteniendo la integridad en los logs históricos de citas.

3.1 Diccionario de Datos de Entidades Principales

Tabla: negocios

Campo	Tipo de Datos	Restricciones	Descripción
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	Identificador único del negocio.
nombre	VARCHAR(150)	NOT NULL	Nombre comercial.
slug	VARCHAR(100)	NOT NULL, UNIQUE	Identificador amigable para URL pública.
logo_url	VARCHAR(255)	NULL	Ruta física local del archivo de imagen.
color_primario	VARCHAR(7)	DEFAULT '#1a365d'	Color institucional hexadecimal.
color_secundario	VARCHAR(7)	DEFAULT '#3182ce'	Color secundario de acento.
whatsapp	VARCHAR(20)	NULL	Teléfono de contacto directo.
created_at	TIMESTAMP	DEFAULT NOW()	Fecha de inserción del registro.
deleted_at	TIMESTAMP	NULL	Control de borrado lógico.

Tabla: citas

Campo	Tipo de Datos	Restricciones	Descripción
id	UUID	PRIMARY KEY	Identificador único de la reserva.
negocio_id	UUID	FOREIGN KEY -> negocios(id)	Vínculo con el negocio emisor.
empleado_id	UUID	FOREIGN KEY -> empleados(id)	Profesional asignado a la cita.
fecha_hora	TIMESTAMP	NOT NULL	Instante temporal exacto de la reserva.
estado	VARCHAR(20)	CHECK (estado IN ('pendiente', 'confirmada', 'completada', 'cancelada', 'no_show'))	Estado actual del ciclo de la cita.
cliente_nombre	VARCHAR(100)	NOT NULL	Nombre del cliente solicitante.
cliente_email	VARCHAR(150)	NOT NULL	Dirección de correo electrónico de destino.
token_operacion	UUID	UNIQUE, NOT NULL	Token seguro para flujos sin login (email).

4. Catálogo de Endpoints de la API RESTful

La comunicación frontend-backend se realiza mediante peticiones HTTP estructuradas con payloads JSON.

4.1 Rutas Públicas (Módulo del Cliente)

Método	Endpoint	Payload de Entrada	Respuesta Exitosa (200/201)
GET	/public/negocios	Ninguno	Lista indexada de todos los negocios activos.
GET	/public/:slug/availability	Query: ?date=YYYY-MM-DD	Matriz de slots horarios de tiempo disponibles libres.
POST	/public/:slug/bookings	{ cliente_nombre, cliente_email, fecha_hora, servicios: [id], empleado_id }	Objeto cita con estado 'pendiente' y token generado.
POST	/public/confirm/:token	Ninguno (parámetro URL)	Confirmación de la cita, cambia el estado a 'confirmada'.

4.2 Rutas de Administración Privada (Guards de Autenticación JWT)

Método	Endpoint	Cabeceras / Payload	Descripción de Operación
POST	/admin/auth/login	{ email, password }	Retorna el access_token JWT expirable firmado.
PATCH	/admin/negocios/:id	Bearer Token + Campos a actualizar	Modifica la descripción, antelación mínima o colores de marca.
POST	/admin/negocios/:id/upload-logo	Multipart FormData (archivo imagen)	Invocación a Multer para subir y guardar localmente el logo.
POST	/admin/bloqueos	{ fecha_inicio, fecha_fin, empleado_id, motivo }	Inhabilita dinámicamente slots del calendario del negocio.

5. Estructura y Organización del Código Fuente

El proyecto mantiene una separación estricta entre la capa de negocio servidor y la vista cliente:

```
AppCitas/  
|-- backend/  
|   |-- src/  
|   |   |-- modules/  
|   |   |   |-- admin/           # Controladores y lógica protegida de administración  
|   |   |   |-- public/         # Endpoints abiertos para la reserva del cliente
```

```
| | | |-- superadmin/      # Gestión global del ecosistema multinegocio
| | | |-- email/          # Motor de envío e logs de correos HTML (Nodemailer)
| | | |-- availability/    # Algoritmos de cálculo de slots libres y solapamientos
| | | |-- file-upload/     # Interceptores Multer para almacenamiento de imágenes
| | | |-- reminders/      # Tareas cron/automatizadas de avisos 24 horas previas
| | | |-- db/             # Configuración de pools de conexión e inicialización
| | | |-- app/            # Módulo raíz del framework NestJS
| | |-- main.js           # Punto de entrada y Bootstrap del servidor
| |-- uploads/logos/      # Servidor estático de imágenes subidas
|-- frontend/
| |-- src/
| | |-- pages/
| | | |-- admin/          # Dashboards de comercios, configuración y agendas
| | | |-- public/         # Landing pages de negocios, calendarios y flujos cliente
| | | |-- superadmin/     # Panel de control de licenciamiento y nuevos comercios
| | |-- services/         # Abstracción de llamadas HTTP mediante Axios/Fetch
| | |-- contexts/        # Proveedores de estado global (sesiones, temas)
| | |-- components/      # Elementos UI (Botones, inputs validados, alertas)
|-- scriptBD.sql          # Esquema DDL completo de la estructura de tablas
|-- seed.sql              # Datos simulados para despliegues iniciales
```

6. Especificación de Subsistemas Críticos

6.1 Motor de Disponibilidad y Bloqueos (availability)

El cálculo de los tramos horarios libres (slots de tiempo) se realiza dinámicamente en el servidor cada vez que un cliente consulta el calendario interactivo. El backend ejecuta una consulta compuesta que cruza:

1. El horario de apertura general definido para el negocio en el día de la semana solicitado.
2. La jornada de trabajo particular e individual asignada al empleado seleccionado.
3. Los tramos de citas ya agendadas con estados `'pendiente'` o `'confirmada'`.
4. Los registros de la tabla de bloqueos (`unavailable times`), que filtran exclusiones completas de jornadas o pausas horarias puntuales.

6.2 Flujo de Notificaciones por Correo Electrónico (email & reminders)

El sistema procesa dos tipologías críticas de emails interactivos utilizando `Nodemailer`:

- **Confirmación Inmediata:** Al realizar una reserva, el servidor genera un token UUID único de operación y despacha un correo multiparte. El cliente de correo renderiza tablas HTML con estilos inline (compatibles con clientes restrictivos como Outlook) con dos botones de acción directa vinculados a los endpoints de aceptación y rechazo automáticos sin autenticación previa. Todos los intentos e incidencias se vuelcan en la tabla de auditoría `email_logs`.

- **Recordatorios Automatizados:** Un proceso asíncrono temporizado evalúa periódicamente la base de datos para identificar citas cuyo campo `fecha_hora` esté exactamente en la ventana de las próximas 24 horas (con un umbral de tolerancia de ± 5 minutos), disparando el aviso preventivo de forma desatendida.

6.3 Pipeline de Subida de Archivos Estáticos (file-upload)

Para garantizar la seguridad del almacenamiento local, la subida de logos mediante `Multer` implementa una validación rígida bicapa: evalúa las firmas mágicas del fichero (MIME-Type restringido a `JPEG, PNG, GIF, WebP`) y aplica un techo de peso binario. Una vez validado, se renombra el archivo mediante un token único para evitar sobrescrituras y se sirve públicamente a través de un middleware de estáticos de NestJS para su previsualización en tiempo real en el frontend de React.